JAIN (Deemed-to-be University)

Department of Computer Science and Engineering - Cybersecurity

PROBLEM-SOLVING ACTIVITY 4
DEBUGGING THE CODE – ADVANCED LOGICAL ANALYSIS AND REFLECTION

NAME-VIJAY.A

USN-25BTRCC055

DEPT-CYBER SECURITY

COURSE-PYTHON PROGRAMMING

Question 1: Operator Precedence, Type Conversion and Conditional Reasoning

A university uses a Python program to evaluate student eligibility for an academic scholarship. A student is eligible only when:

- All three subject marks are at least 40.
- The average of the three marks is at least 75.
- The attendance percentage is at least 75.

The following program is intended to determine eligibility and calculate the average.

```
def evaluate(m1, m2, m3, attendance):
average = m1 + m2 + m3 / 3

    if m1 >= 40 and m2 >= 40 or m3 >= 40:
if average >= 75 and attendance >= 75:
return average, "Eligible"      return average,
"Not Eligible"


data = input("Enter marks and attendance: ").split()

result = evaluate(
    data[0], data[1], data[2], data[3]
)
```

```
print("Average:", result[0])
print("Status:", result[1])
```

Error diagnosis

(m1, m2, m3, attendance): (incorrect average calculation, In this only m3 is divided by 3 not M1+M2+M3 is /3.

if m1 >= 40 and m2 >= 40 or m3 >= 40: (incorrect Boolean condition, The "OR" operator condition to become true the provided first two marks do not both pass)

data = input("Enter marks and attendance: ").split() (the split() function returns string values directly it cannot compare strings with integer using >=)

Code correction

```
def evaluate(m1, m2, m3, attendance):

    average = (m1 + m2 + m3) / 3

    if m1 >= 40 and m2 >= 40 and m3 >= 40:
        if average >= 75 and attendance >= 75:
            return average, "Eligible"

    return average, "Not Eligible"


data = list(map(float, input("Enter marks and attendance: ").sp

result = evaluate(
    data[0], data[1], data[2], data[3]
)

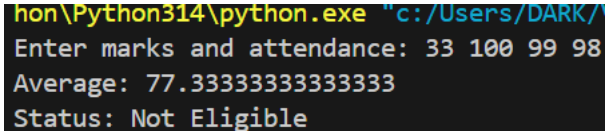
print("Average:", result[0])
print("Status:", result[1])
```

Testing and output analysis

GIVEN CONDITION- Test with 80 70 90 85.

```
hon\Python314\python.exe "c:/Users/DARK/
Enter marks and attendance: 80 70 90 85
Average: 80.0
Status: Eligible
```

Then test a case where one subject mark is below 40



but the other marks and attendance satisfy the remaining conditions. Explain why eligibility changes

WHY ELIGIBILITY CHNAGES BECAUSE

AVERAGE AND ATTENDANCE SATISFY THE CONDITION,BUT THE FIRST MARKS IN THE MARKS BELOW 40 IS NOT ELIGIBLE FOR THE SCHOLARSHIP

Individual reflection

data = input("Enter marks and attendance: ").split()

The most difficult error was the Boolean condition because it did not check all three subject marks correctly. I corrected it using and and added brackets to calculate the average properly. I learned that operator precedence decides the order of calculation and that all conditions must be true for a student to be eligible.

Question 2: Nested Loops, Loop Control and Prime Number Generation

A student is developing a program to generate all prime numbers within an inclusive range. The following implementation contains multiple errors

```
def generate_primes(start, end):
    primes = []

    for number in range(start,
end):         is_prime = True

        if number <
2:
is_prime = False
```

for divisor in
range(2,int(number**0.5)):
if number % divisor == 0:
is_prime =
False
continue

if is_prime:

primes.append(number)

return primes

print(generate_primes(1, 30))

Error diagnosis

range(start, end + 1) (range error)

int(number ** 0.5) (This may cause composite numbers like 9 to be wrongly classified as prime.)

continue (We should use break)

Code correction

```
from math import isqrt
def generate_primes(start, end):
    primes = []
    for number in range(start, end + 1):
        if number < 2:
            continue
        is_prime = True
        for divisor in range(2, isqrt(number) + 1):
            if number % divisor == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(number)
    return primes
print(generate_primes(1, 30))
```

Testing and output analysis

GIVEN CONDITION- Determine the correct output for generate_primes(1, 30)

```
hon\Python314\python.exe "c:/Users/DARK/
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Also test generate_primes(2, 2)

```
hon\Pyt
[2]
```

generate_primes(14, 16)

```
hon\Pyth
[]
```

Individual reflection

I used break to stop unnecessary checks after finding a factor. The test case generate_primes(2, 2) helped me check whether the ending number was included correctly. I learned that loop control and proper testing are important for finding logical errors.

Question 3: Recursion, Base Cases and Function Return Values

A college wants to calculate the sum of all digits in a positive integer. For example, the sum of the digits of 5824 is 5 + 8 + 2 + 4 = 19. The following recursive program is intended to calculate this sum.

```
def digit_sum(number):
if number < 10:
return number

    digit = number % 10
digit_sum(number // 10)
return digit


number = 5824
print("Sum of digits:", digit_sum(number))
```

Error diagnosis

digit_sum(number // 10) (but does not store or use its returned value)

return statement (incorrect return statement)

Code correction

```
def digit_sum(number):
    if number < 10:
        return number
    digit = number % 10
    return digit + digit_sum(number // 10)
number = 5824
print("Sum of digits:", digit_sum(number))
```

Testing and output analysis

Trace the corrected function for 5824 and determine the output

```
hon\Python314\python.exe
Sum of digits: 19
```

Test it again with 7 and 0, explaining how the base case behaves.

```
hon\Python314\python
Sum of digits: 7
```

c

```
hon\Python314\python.exe
Sum of digits: 0
```

Since 0 is less than 10, the base case returns 0.

Individual Reflection

I learned that making a recursive call is not enough; we must use its returned value to get the correct result.

Question 4: Lists, Mutation, Comprehensions and Sorting

A college processes marks obtained by students in a programming assessment. The program must:

1. Remove all marks below 40.
2. Remove duplicate marks.
3. Sort the remaining marks in ascending order.
4. Identify marks greater than or equal to 60.
5. Display the three highest qualifying marks, or all qualifying marks if fewer than three exist.

The following program is intended to perform these operations.

```
marks = [35, 60, 45, 80, 35, 55, 60, 95, 40, 80]

for mark in marks:     if
mark < 40:
marks.remove(mark)

unique_marks = list(set(marks))
unique_marks.sort(reverse=True)

qualified = [     mark for mark in
unique_marks
    if mark > 60
] top_three =

qualified[:3]

print("Processed marks:", unique_marks)
print("Qualified marks:", qualified)
print("Top three:", top_three)
```

Error Diagnosis

reverse=True (sorts marks in descending order, but the requirement is ascending order)

mark > 60 (it should be mark >= 60)

The program should display the three highest qualifying marks in descending order.

Code Correction

```
marks = [35, 60, 45, 80, 35, 55, 60, 95, 40, 80]

# Remove marks below 40 and duplicates
unique_marks = sorted(set(mark for mark in marks if mark >= 40))

# Identify qualifying marks
qualified = [mark for mark in unique_marks if mark >= 60]

# Display the three highest marks
top_three = sorted(qualified, reverse=True)[:3]

print("Processed marks:", unique_marks)
print("Qualified marks:", qualified)
print("Top three:", top_three)
```

Testing and Output Analysis

GIVEN CONDITION- Determine the correct processed list, qualifying marks, and top three qualifying marks. Explain how your implementation handles duplicates and marks exactly equal to 40 and 60.

```
hon\Python314\python.exe "c:/Users/DARK/V
Processed marks: [40, 45, 55, 60, 80, 95]
Qualified marks: [60, 80, 95]
Top three: [95, 80, 60]
```

Individual Reflection

I verified the output by checking the ascending order and testing marks exactly equal to 40 and 60.

Question 5: Dictionaries, Sets, Nested Data and Data Consistency

A university maintains course registrations in a dictionary. Identify all students who share at least two distinct courses with the target student, Arun. The output must contain each qualifying student's name as the key and a sorted list of common courses as the value. The target student must not appear in the output.

```
registrations = {
    "Arun": ["Python", "Java", "SQL", "Python"],
    "Bala": ["Python", "C", "SQL"],
    "Charan": ["Java", "Python", "SQL"],
    "Divya": ["Python", "Java"],
    "Ezhil": ["C", "Java", "Java"]
}

target = "Arun"
common_students = {}

for student, courses in registrations.items():     if
student != target:         common =
set(registrations[target]) & set(courses)

        if len(common) > 2:
common_students[student] = list(common)
print("Common courses:", common_students)
```

Error diagnosis

len(common) > 2 (it should be len(common) >= 2)

Some students have registered for the same course more than once. Using set() removes duplicates and counts only distinct courses.

Sets do not maintain alphabetical order. We should use sorted() to display the common courses in alphabetical order.

Code correction.

```python
registrations = {
    "Arun": ["Python", "Java", "SQL", "Python"],
    "Bala": ["Python", "C", "SQL"],
    "Charan": ["Java", "Python", "SQL"],
    "Divya": ["Python", "Java"],
    "Ezhil": ["C", "Java", "Java"]
}
target = "Arun"
common_students = {}
for student, courses in registrations.items():
    if student != target:
        common = set(registrations[target]) & set(courses)

        if len(common) >= 2:
            common_students[student] = sorted(common)
print("Common courses:", common_students)
```

Testing and output.

```
hon\Python314\python.exe "c:/Users/DARK/Videos/python course/assignment.py"
Common courses: {'Bala': ['Python', 'SQL'], 'Charan': ['Java', 'Python', 'SQL'],
'Divya': ['Java', 'Python']}
```

Individual reflection

I used sets to find common courses easily and remove duplicate registrations. I used the >= 2 condition to include students sharing exactly two courses. I used sorted() to arrange the courses alphabetically and make the output consistent.